Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Bases de données relationnelles, jointures SQL et agrégations

Synthèse exécutive : Ce cours aborde les mécanismes fondamentaux du langage SQL pour croiser des données issues de multiples tables relationnelles via les jointures (INNER JOIN, LEFT JOIN) et structurer les résultats par groupes au moyen des clauses GROUP BY et HAVING. L'objectif est de permettre la construction de requêtes complexes tout en évitant les erreurs classiques de produits cartésiens involontaires ou de filtres incorrects sur données agrégées.

Les Jointures Relationnelles : INNER JOIN vs OUTER JOIN

Dans un SGBDR (Système de Gestion de Base de Données Relationnel), la normalisation impose la séparation des entités métiers dans des tables distinctes. La jointure est l'opération algébrique permettant d'associer deux relations à partir d'un critère d'équivalence entre deux attributs (généralement la clé primaire d'une table et la clé étrangère d'une autre). L'évaluation de la clause JOIN s'effectue dans le bloc FROM avant tout filtrage par la clause WHERE. Il est crucial d'expliciter le critère de liaison dans la clause 'ON' pour éviter de générer un produit cartésien très coûteux en mémoire et en temps d'exécution.

- INNER JOIN (Jointure Interne) : Restitue uniquement les combinaisons de n-uplets pour lesquelles la condition de jointure est vraie dans les deux tables.
- LEFT JOIN (Jointure Externe Gauche) : Conserve la totalité des enregistrements de la table de gauche, complétés par les données de la table de droite ou par des valeurs NULL si aucune correspondance n'existe.
- Clé Étrangère (FK) & Clé Primaire (PK) : Pivot de l'équijointure reliant un attribut d'une table source au produit de la table cible.
- Produit Cartésien (CROSS JOIN) : Résultat d'une jointure sans prédicat d'association, combinant chaque ligne de la table A avec toutes les lignes de la table B.

Cas concret enseignant : *Le professeur compare la jointure interne (INNER JOIN) à l'accès d'un bâtiment sécurisé : seuls les étudiants possédant une carte valide associée à une inscription effective entrent. Pour le LEFT JOIN, l'analogie donnée est celle de la liste d'appel globale de l'IUT de Bayonne : tous les étudiants inscrits apparaissent sur la liste, même si certains n'ont pas encore de note saisie (la valeur de la note affichera alors 'NULL').*

Agrégation de Données et Regroupement : GROUP BY & HAVING

L'agrégation permet de réaliser des opérations statistiques synthétiques sur des partitions de la base de données. La clause GROUP BY rassemble les tuples ayant la même valeur pour un ensemble de colonnes données. Dès qu'un GROUP BY est présent, les colonnes sélectionnées dans le SELECT doivent obligatoirement soit figurer dans le GROUP BY, soit être intégrées à une fonction d'agrégation. La clause WHERE ne pouvant pas évaluer le résultat d'une fonction d'agrégation (car elle intervient AVANT le regroupement), le langage SQL introduit la clause HAVING pour appliquer des filtres postérieurs à l'agrégation.

- Fonctions d'Agrégation : Fonctions scalaires calculant une valeur unique à partir d'un ensemble de n-uplets (COUNT, SUM, AVG, MIN, MAX).
- GROUP BY : Clause subdivisant les lignes de la table en sous-groupes homogènes partageant les mêmes valeurs sur les attributs spécifiés.
- HAVING : Prédicat de filtrage s'appliquant spécifiquement sur les groupes créés et les résultats des fonctions d'agrégation.
- Ordre logique d'exécution SQL : FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Cas concret enseignant : *L'enseignant illustre le GROUP BY en prenant le tri physique des copies d'examen : on regroupe d'abord les copies par département (Informatique, GEA, TC), ce qui constitue la clause GROUP BY. Ensuite, pour chaque paquet, on calcule la moyenne des notes (fonction AVG). Enfin, si le directeur souhaite afficher uniquement les départements dont la moyenne est supérieure à 12/20, c'est la clause HAVING qui élimine les paquets ne respectant pas cette condition.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il impossible d'écrire WHERE AVG(note) > 10 dans une requête SQL ?
Réponse du professeur : Parce que la clause WHERE s'exécute séquentiellement avant le regroupement (GROUP BY) et l'agrégation. Au moment où le WHERE est évalué, les lignes individuelles n'ont pas encore été regroupées pour calculer une moyenne. C'est précisément pour cela que la clause HAVING existe : elle s'exécute après le GROUP BY.

Question étudiant : Que se passe-t-il si j'oublie d'inclure une colonne du SELECT dans le GROUP BY ?
Réponse du professeur : Le SGBD renvoie une erreur de syntaxe (ou un comportement non déterministe selon le moteur), car SQL ne saurait pas quelle valeur afficher pour cette colonne parmi toutes celles qui ont été regroupées. Toute colonne projetée dans le SELECT doit être soit dans le GROUP BY, soit sous une fonction d'agrégation.

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données de l'IUT composée des tables Etudiant(id_etud, nom, id_promo) et Note(id_note, id_etud, matiere, note). Écrire la requête SQL qui affiche le nom de l'étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants ayant au moins 2 notes dans cette matière et une moyenne stricte supérieure à 12, triés par moyenne décroissante.

Résolution pas à pas :
1. Déterminer les tables nécessaires : Etudiant (E) et Note (N). 2. Réaliser l'équijointure entre les tables : E.id_etud = N.id_etud. 3. Appliquer le filtre individuel sur la matière dans la clause WHERE : N.matiere = 'R1.05'. 4. Regrouper par identifiant et nom d'étudiant : GROUP BY E.id_etud, E.nom. 5. Filtrer les groupes formés dans la clause HAVING avec deux conditions : COUNT(N.id_note) >= 2 AND AVG(N.note) > 12. 6. Projeter les colonnes désirées dans le SELECT : E.nom, AVG(N.note) AS moyenne. 7. Ordonner les résultats avec la clause ORDER BY : ORDER BY moyenne DESC.

Résultat : SELECT E.nom, AVG(N.note) AS moyenne FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud WHERE N.matiere = 'R1.05' GROUP BY E.id_etud, E.nom HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12 ORDER BY moyenne DESC;

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'application entre la clause WHERE et la clause HAVING ?

R : La clause WHERE filtre les lignes individuelles avant l'agrégation, tandis que la clause HAVING filtre les groupes formés après l'agrégation (et peut contenir des fonctions d'agrégation).

Q : Quel est le résultat d'un LEFT JOIN si aucune ligne de la table de droite ne correspond à un enregistrement de la table de gauche ?

R : Toutes les lignes de la table de gauche sont conservées, et les colonnes issues de la table de droite sont complétées par la valeur NULL.

Q : Si une table A contient 10 lignes et une table B contient 5 lignes, combien de lignes retourne un produit cartésien (CROSS JOIN) entre A et B ?

*R : Le produit cartésien retourne 10 * 5 = 50 lignes.*